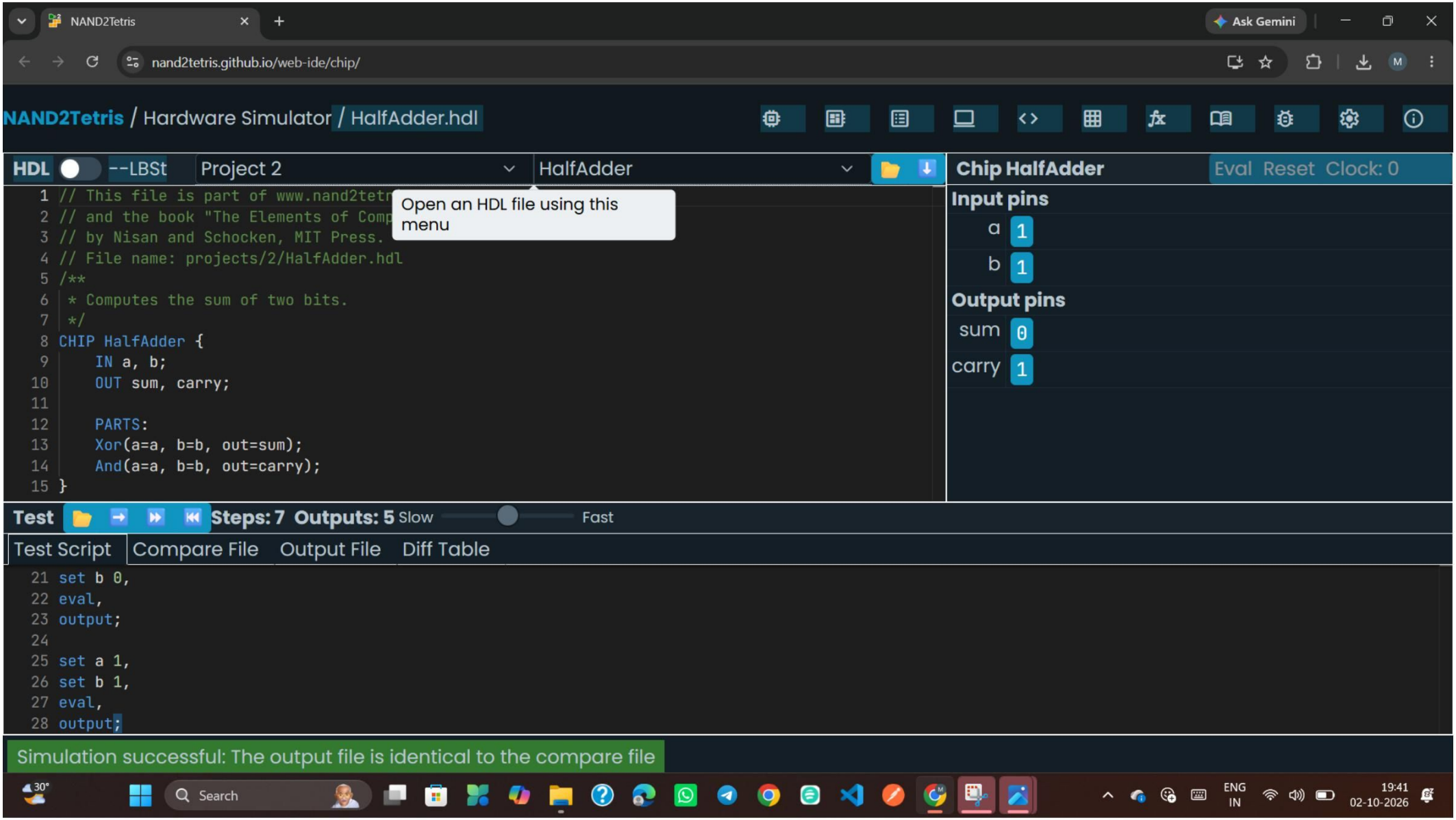
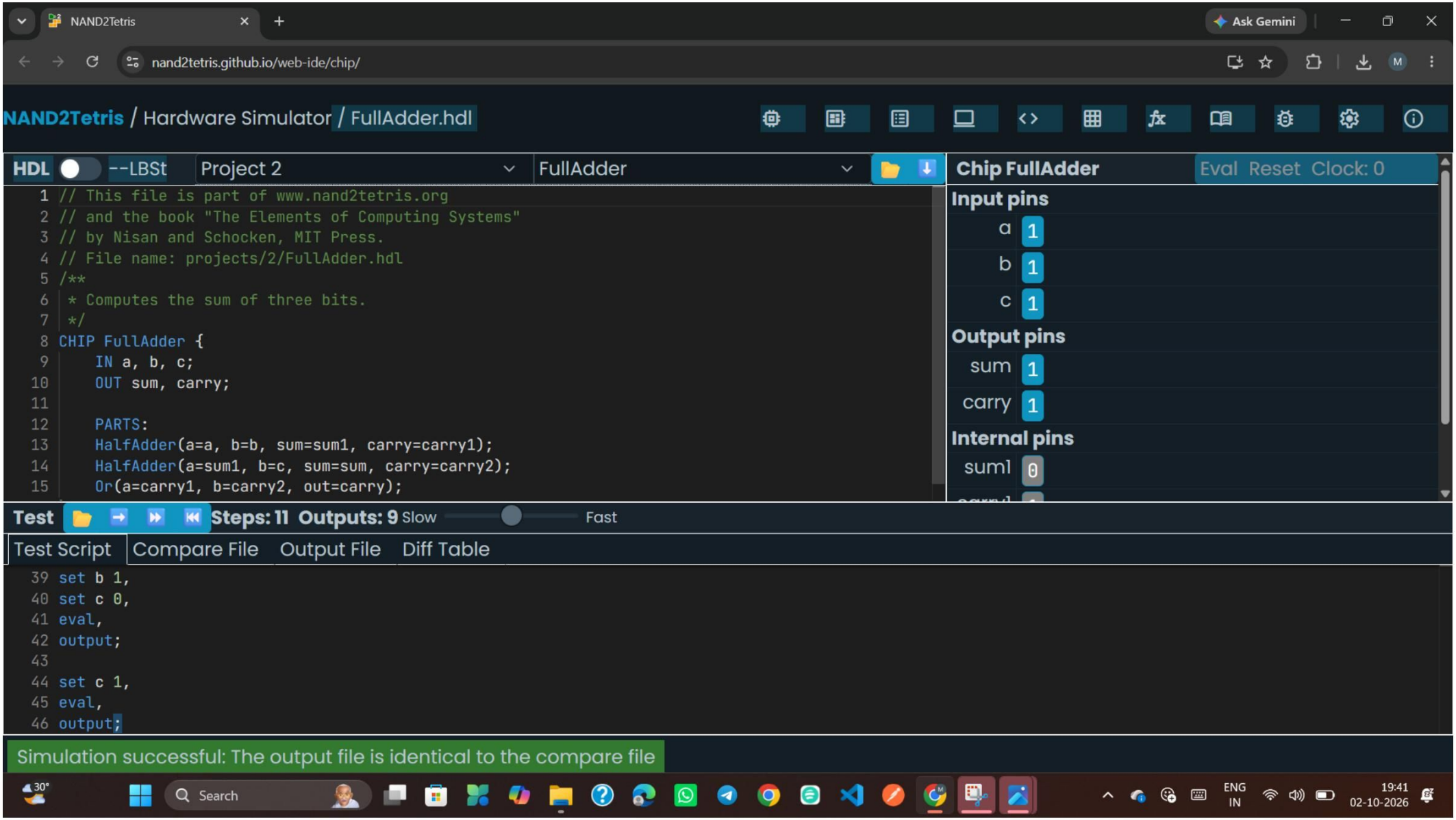
# ASSIGNMENT

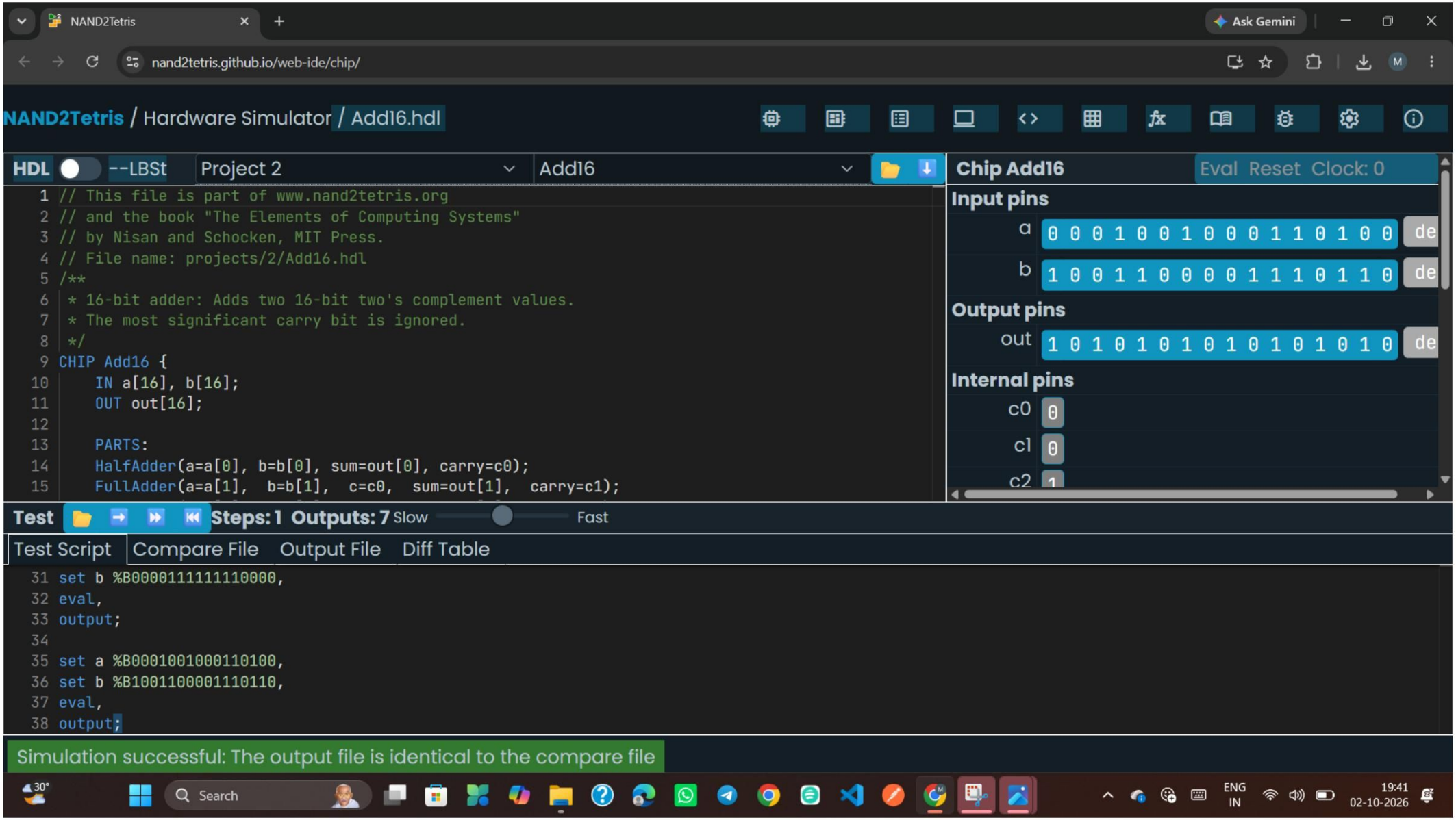
Name: Mukul Kumar

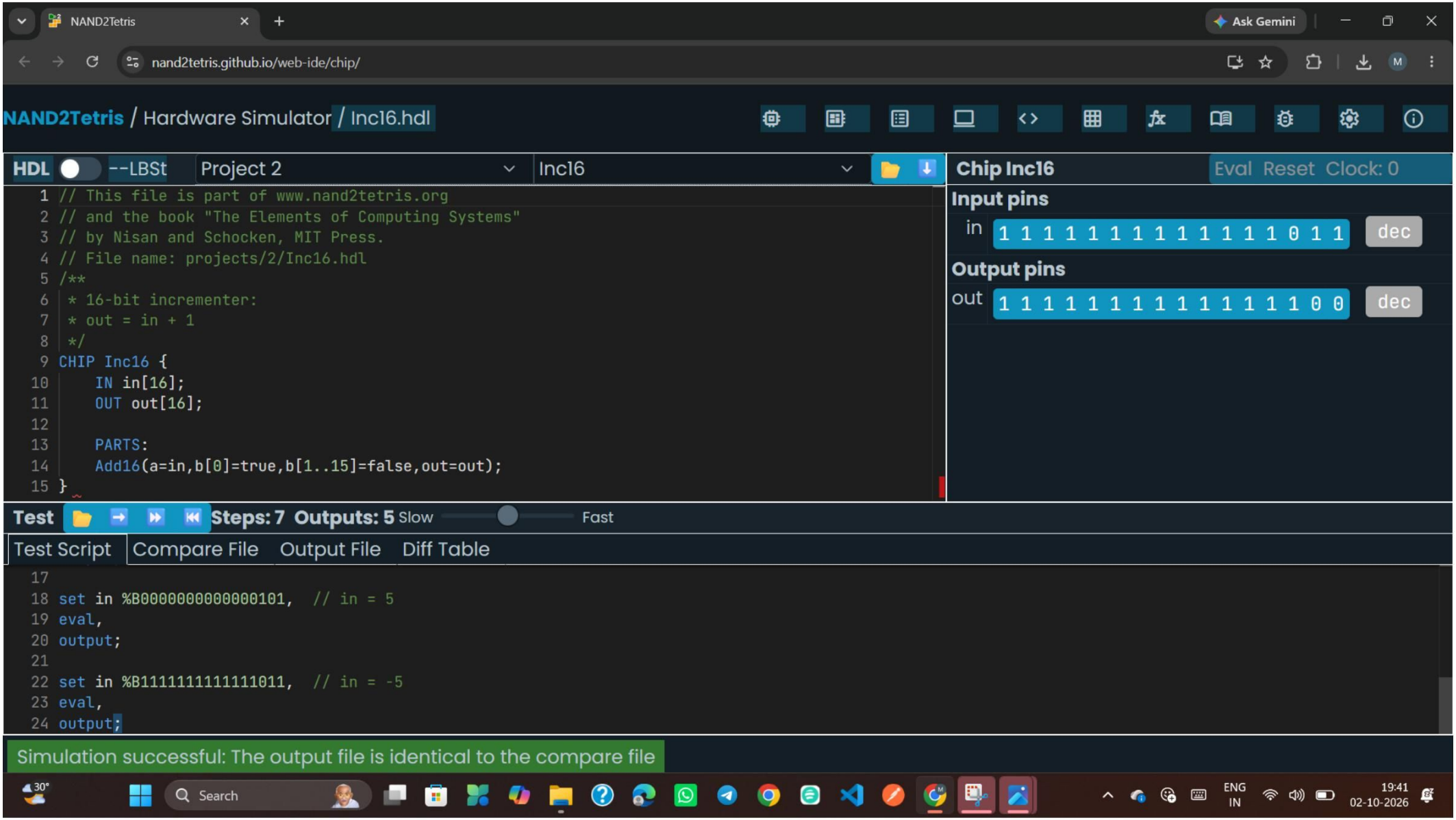
Roll: 2501010127

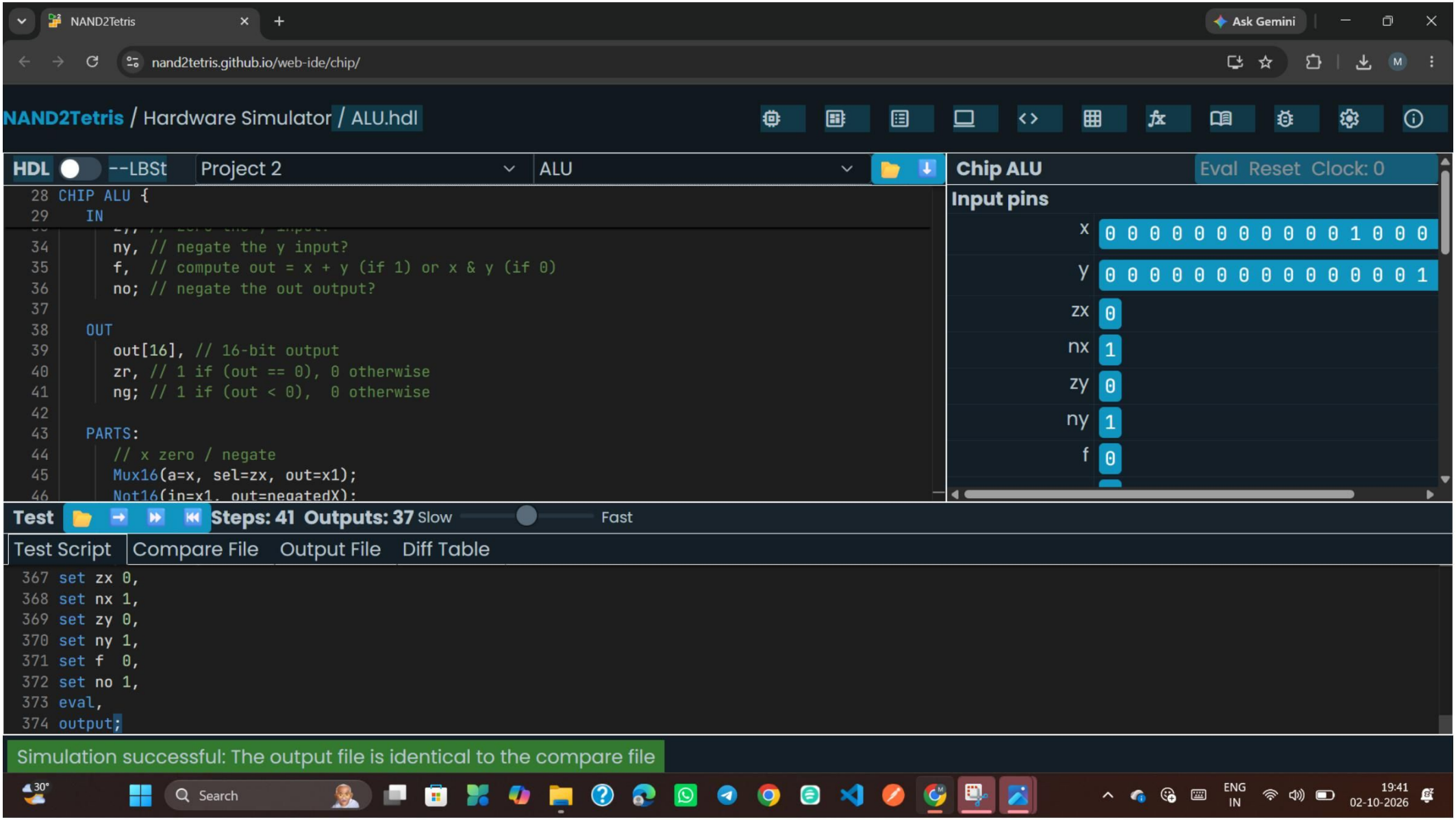
Sec: C


NAND2Tetris
Ask Gemini
nand2tetris.github.io/web-ide/chip/
NAND2Tetris / Hardware Simulator / HalfAdder.hdl
HDL
--LBSt
Project 2
HalfAdder
Open an HDL file using this menu
1 // This file is part of www.nand2tetr
2 // and the book "The Elements of Comp
3 // by Nisan and Schocken, MIT Press.
4 // File name: projects/2/HalfAdder.hdl
5 /**
6 * Computes the sum of two bits.
7 */
8 CHIP HalfAdder {
9 IN a, b;
10 OUT sum, carry;
11
12 PARTS:
13 Xor(a=a, b=b, out=sum);
14 And(a=a, b=b, out=carry);
15 }
Chip HalfAdder
Eval Reset Clock: 0
Input pins
a 1
b 1
Output pins
sum 0
carry 1
Test
Steps: 7 Outputs: 5 Slow Fast
Test Script
Compare File
Output File
Diff Table
21 set b 0,
22 eval,
23 output;
24
25 set a 1,
26 set b 1,
27 eval,
28 output;
Simulation successful: The output file is identical to the compare file
Search
ENG
IN
19:41
02-10-2026


NAND2Tetris
Ask Gemini
nand2tetris.github.io/web-ide/chip/
NAND2Tetris / Hardware Simulator / FullAdder.hdl
HDL --LBSt Project 2 FullAdder
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/2/FullAdder.hdl
/**
* Computes the sum of three bits.
*/
CHIP FullAdder {
IN a, b, c;
OUT sum, carry;
PARTS:
HalfAdder(a=a, b=b, sum=sum1, carry=carry1);
HalfAdder(a=sum1, b=c, sum=sum, carry=carry2);
Or(a=carry1, b=carry2, out=carry);
Chip FullAdder
Eval Reset Clock: 0
Input pins
a 1
b 1
c 1
Output pins
sum 1
carry 1
Internal pins
sum1 0
Test Steps: 11 Outputs: 9 Slow Fast
Test Script
Compare File
Output File
Diff Table
set b 1,
set c 0,
eval,
output;
set c 1,
eval,
output;
Simulation successful: The output file is identical to the compare file
Search
ENG
IN
19:41
02-10-2026


NAND2Tetris
Ask Gemini
nand2tetris.github.io/web-ide/chip/
NAND2Tetris / Hardware Simulator / Add16.hdl
HDL --LBSt Project 2 Add16
1 // This file is part of www.nand2tetris.org
2 // and the book "The Elements of Computing Systems"
3 // by Nisan and Schocken, MIT Press.
4 // File name: projects/2/Add16.hdl
5 /**
6 * 16-bit adder: Adds two 16-bit two's complement values.
7 * The most significant carry bit is ignored.
8 */
9 CHIP Add16 {
10 IN a[16], b[16];
11 OUT out[16];
12
13 PARTS:
14 HalfAdder(a=a[0], b=b[0], sum=out[0], carry=c0);
15 FullAdder(a=a[1], b=b[1], c=c0, sum=out[1], carry=c1);
Chip Add16 Eval Reset Clock: 0
Input pins
a 0 0 0 1 0 0 1 0 0 0 1 1 0 1 0 0
b 1 0 0 1 1 0 0 0 0 1 1 1 0 1 1 0
Output pins
out 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0
Internal pins
c0 0
c1 0
c2 1
Test Steps: 1 Outputs: 7 Slow Fast
Test Script Compare File Output File Diff Table
31 set b %B0000111111110000,
32 eval,
33 output;
34
35 set a %B0001001000110100,
36 set b %B1001100001110110,
37 eval,
38 output;
Simulation successful: The output file is identical to the compare file
Search
ENG IN
19:41 02-10-2026


NAND2Tetris
Ask Gemini
nand2tetris.github.io/web-ide/chip/
NAND2Tetris / Hardware Simulator / Inc16.hdl
HDL --LBSt Project 2 Inc16
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/2/Inc16.hdl
/**
* 16-bit incrementer:
* out = in + 1
*/
CHIP Inc16 {
IN in[16];
OUT out[16];
PARTS:
Add16(a=in,b[0]=true,b[1..15]=false,out=out);
}
Chip Inc16 Eval Reset Clock: 0
Input pins
in 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 dec
Output pins
out 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 dec
Test Steps: 7 Outputs: 5 Slow Fast
Test Script Compare File Output File Diff Table
set in %B0000000000000101, // in = 5
eval,
output;
set in %B1111111111111011, // in = -5
eval,
output;
Simulation successful: The output file is identical to the compare file
Search
ENG IN
19:41 02-10-2026


NAND2Tetris
Ask Gemini
nand2tetris.github.io/web-ide/chip/
NAND2Tetris / Hardware Simulator / ALU.hdl
HDL --LBSt Project 2 ALU
CHIP ALU {
IN
ny, // negate the y input?
f, // compute out = x + y (if 1) or x & y (if 0)
no; // negate the out output?
OUT
out[16], // 16-bit output
zr, // 1 if (out == 0), 0 otherwise
ng; // 1 if (out < 0), 0 otherwise
PARTS:
// x zero / negate
Mux16(a=x, sel=zx, out=x1);
Chip ALU
Eval Reset Clock: 0
Input pins
x 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0
y 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1
zx 0
nx 1
zy 0
ny 1
f 0
Test
Steps: 41 Outputs: 37 Slow Fast
Test Script
Compare File
Output File
Diff Table
set zx 0,
set nx 1,
set zy 0,
set ny 1,
set f 0,
set no 1,
eval,
output;
Simulation successful: The output file is identical to the compare file
Search
ENG
IN
19:41
02-10-2026

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/2/Add16.hdl
/**
 * 16-bit adder: Adds two 16-bit two's complement values.
 * The most significant carry bit is ignored.
 */
CHIP Add16 {
    IN a[16], b[16];
    OUT out[16];

    PARTS:
    HalfAdder(a=a[0], b=b[0], sum=out[0], carry=c0);
    FullAdder(a=a[1],  b=b[1],  c=c0,  sum=out[1],  carry=c1);
    FullAdder(a=a[2],  b=b[2],  c=c1,  sum=out[2],  carry=c2);
    FullAdder(a=a[3],  b=b[3],  c=c2,  sum=out[3],  carry=c3);
    FullAdder(a=a[4],  b=b[4],  c=c3,  sum=out[4],  carry=c4);
    FullAdder(a=a[5],  b=b[5],  c=c4,  sum=out[5],  carry=c5);
    FullAdder(a=a[6],  b=b[6],  c=c5,  sum=out[6],  carry=c6);
    FullAdder(a=a[7],  b=b[7],  c=c6,  sum=out[7],  carry=c7);
    FullAdder(a=a[8],  b=b[8],  c=c7,  sum=out[8],  carry=c8);
    FullAdder(a=a[9],  b=b[9],  c=c8,  sum=out[9],  carry=c9);
    FullAdder(a=a[10], b=b[10], c=c9,  sum=out[10], carry=c10);
    FullAdder(a=a[11], b=b[11], c=c10, sum=out[11], carry=c11);
    FullAdder(a=a[12], b=b[12], c=c11, sum=out[12], carry=c12);
    FullAdder(a=a[13], b=b[13], c=c12, sum=out[13], carry=c13);
    FullAdder(a=a[14], b=b[14], c=c13, sum=out[14], carry=c14);
    FullAdder(a=a[15], b=b[15], c=c14, sum=out[15], carry=overflow);
}
```

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/2/Inc16.hdl
/**
 * 16-bit incrementer:
 * out = in + 1
 */
CHIP Inc16 {
    IN in[16];
    OUT out[16];

    PARTS:
    Add16(a=in,b[0]=true,b[1..15]=false,out=out);
}
```

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/2/HalfAdder.hdl
/**
 * Computes the sum of two bits.
 */
CHIP HalfAdder {
    IN a, b;
    OUT sum, carry;

    PARTS:
    Xor(a=a, b=b, out=sum);
    And(a=a, b=b, out=carry);
}
```

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/2/FullAdder.hdl
/**
 * Computes the sum of three bits.
 */
CHIP FullAdder {
    IN a, b, c;
    OUT sum, carry;

    PARTS:
    HalfAdder(a=a, b=b, sum=sum1, carry=carry1);
    HalfAdder(a=sum1, b=c, sum=sum, carry=carry2);
    Or(a=carry1, b=carry2, out=carry);
}
```

```
	// negate the output?
	Not16(in=result, out=negatedResult);
	Mux16(a=result, b=negatedResult, sel=no, out=out, out[15]=firstOut,
out[0..7]=finalLeft, out[8..15]=finalRight);

	// output == 0 (zr)
	Or8Way(in=finalLeft, out=zrl);
	Or8Way(in=finalRight, out=zrr);
	Or(a=zrl, b=zrr, out=nzr);
	Not(in=nzr, out=zr);

	// output < 0 (ng)
	And(a=firstOut, b=true, out=ng);
}
```

```
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/2/ALU.hdl
/**
 * ALU (Arithmetic Logic Unit):
 * Computes out = one of the following functions:
 *                0, 1, -1,
 *                x, y, !x, !y, -x, -y,
 *                x + 1, y + 1, x - 1, y - 1,
 *                x + y, x - y, y - x,
 *                x & y, x | y
 * on the 16-bit inputs x, y,
 * according to the input bits zx, nx, zy, ny, f, no.
 * In addition, computes the two output bits:
 * if (out == 0) zr = 1, else zr = 0
 * if (out < 0)  ng = 1, else ng = 0
 */
// Implementation: Manipulates the x and y inputs
// and operates on the resulting values, as follows:
// if (zx == 1) sets x = 0        // 16-bit constant
// if (nx == 1) sets x = !x       // bitwise not
// if (zy == 1) sets y = 0        // 16-bit constant
// if (ny == 1) sets y = !y       // bitwise not
// if (f == 1)  sets out = x + y  // integer 2's complement addition
// if (f == 0)  sets out = x & y  // bitwise and
// if (no == 1) sets out = !out   // bitwise not
CHIP ALU {
    IN
        x[16], y[16],  // 16-bit inputs
        zx, // zero the x input?
        nx, // negate the x input?
        zy, // zero the y input?
        ny, // negate the y input?
        f,  // compute out = x + y (if 1) or x & y (if 0)
        no; // negate the out output?

    OUT
        out[16], // 16-bit output
        zr, // 1 if (out == 0), 0 otherwise
        ng; // 1 if (out < 0),  0 otherwise

    PARTS:
        // x zero / negate
        Mux16(a=x, sel=zx, out=x1);
        Not16(in=x1, out=negatedX);
        Mux16(a=x1, b=negatedX, sel=nx, out=x2);

        // y zero / negate
        Mux16(a=y, sel=zy, out=y1);
        Not16(in=y1, out=negatedY);
        Mux16(a=y1, b=negatedY, sel=ny, out=y2);

        // x + y OR x & y
        Add16(a=x2, b=y2, out=added);
        And16(a=x2, b=y2, out=xyAnd);
        Mux16(a=xyAnd, b=added, sel=f, out=result);
```